classic arcade Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is OchsSoft/classic-arcade at 21cf8599dd68875ebd2042cf8e5e0f88c418cdac. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand the project 3
Distribution architecture and data ownership 4

Use and develop the project

Local setup on NixOS and other platforms 7
Player journeys and normal close 8
Maintaining releases and documentation 9

Diagnose improve and verify

Known gaps and recovery 11
Source reference 12

Understand the project

The product and this repository

This is the public download and installation repository for OchsSoft Classic Arcade. The root README explicitly says that game and website source are maintained privately. This checkout therefore cannot be used to change game logic, rebuild executables, inspect save code or run source tests. Its development work is release presentation, installation documentation and the integrity of published assets.

A release asset is a packaged file attached to a GitHub release rather than a normal source file. The repository links the latest release and named platform archives. The links using `releases/latest` move when a newer release becomes latest; they are convenient entry points, not immutable version evidence. For a reproducible support report, record the actual downloaded release tag, asset name, checksum and platform.

The reader has two roles. As a player, download the appropriate archive, verify it where a checksum is supplied, extract the whole program folder and launch the platform entry point. As a maintainer, check that documentation matches the actual release, investigate reports without exposing private source, and coordinate any executable change in the separate source repository's own workflow.

Platform promises in the committed README

Windows uses `ClassicArcade-windows.zip` and `ClassicArcade.exe`. Linux x86_64 uses `ClassicArcade-linux-x86_64.tar.gz`, a SHA-256 checksum companion, and `./ClassicArcade/ClassicArcade`. The stated Linux requirement is a graphical desktop with glibc 2.35 or newer; NixOS uses `steam-run` around the launcher. macOS uses `ClassicArcade-macos.zip`, a checksum, and `ClassicArcade.app` copied into Applications.

The README says the macOS app is Developer ID signed, notarized and stapled, includes Python and dependencies, supports Apple Silicon and Intel, and does not require installing Python or Rosetta. These are the release documentation's assertions. This handbook did not download or inspect a current binary and does not claim a fresh signing, architecture or NixOS runtime verification.

What is intentionally unknown here

There is no committed API, game-loop source, settings schema, Linux data-path contract, build workflow or automated test harness. A maintainer should ask the source/release owner for those facts when needed rather than fill the public guide with guesses. The public documentation must remain useful on its own while accurately marking that evidence boundary.

Distribution architecture and data ownership

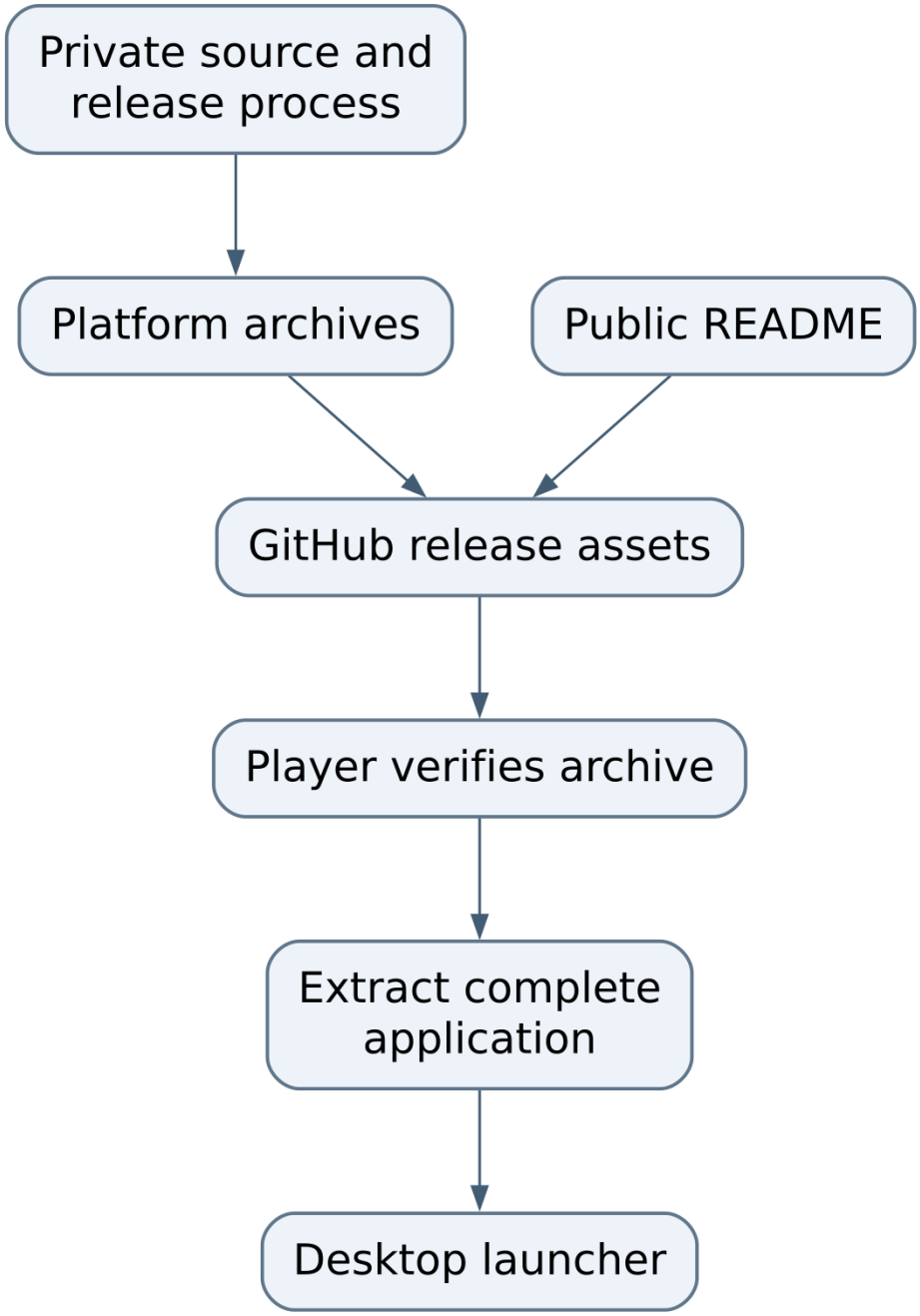


Figure 1 System flow

Only the README and documentation are inspectable implementation in this repository. The source/build process is an external boundary, not a runnable subdirectory here. A player needs the release archive, not a clone of this repository. Conversely, a source maintainer needs the source project; downloading the program is not a substitute for editable source.

Package layout is a contract

The Linux command expects a `ClassicArcade` directory containing an executable named `ClassicArcade`. Windows expects the extracted executable and its neighboring files to stay together. macOS expects the application bundle to remain intact. Moving only a launcher out of a bundled application can break dynamic-library, Python-runtime or resource lookup. The README specifically tells Windows users to keep the program folder together; apply the same conservative approach to all packaged platforms unless a release explicitly says otherwise.

A checksum is a digest of exact archive bytes. It detects accidental corruption and mismatch against a published companion file. It does not independently establish publisher identity if both files came from an untrusted source. Download from the repository's actual release page and record the version. Do not edit an archive and then continue describing its original checksum as valid.

Installed files versus personal data

The README explicitly locates macOS settings and statistics at `~/Library/Application Support/OchsSoftClassicArcade`, outside the application bundle. Replacing the app should therefore preserve that directory. The exact filenames and schemas are not documented in this checkout. The Windows and Linux personal-data locations are not specified in the root README; do not assume they are inside the installation directory or use a guessed cleanup command.

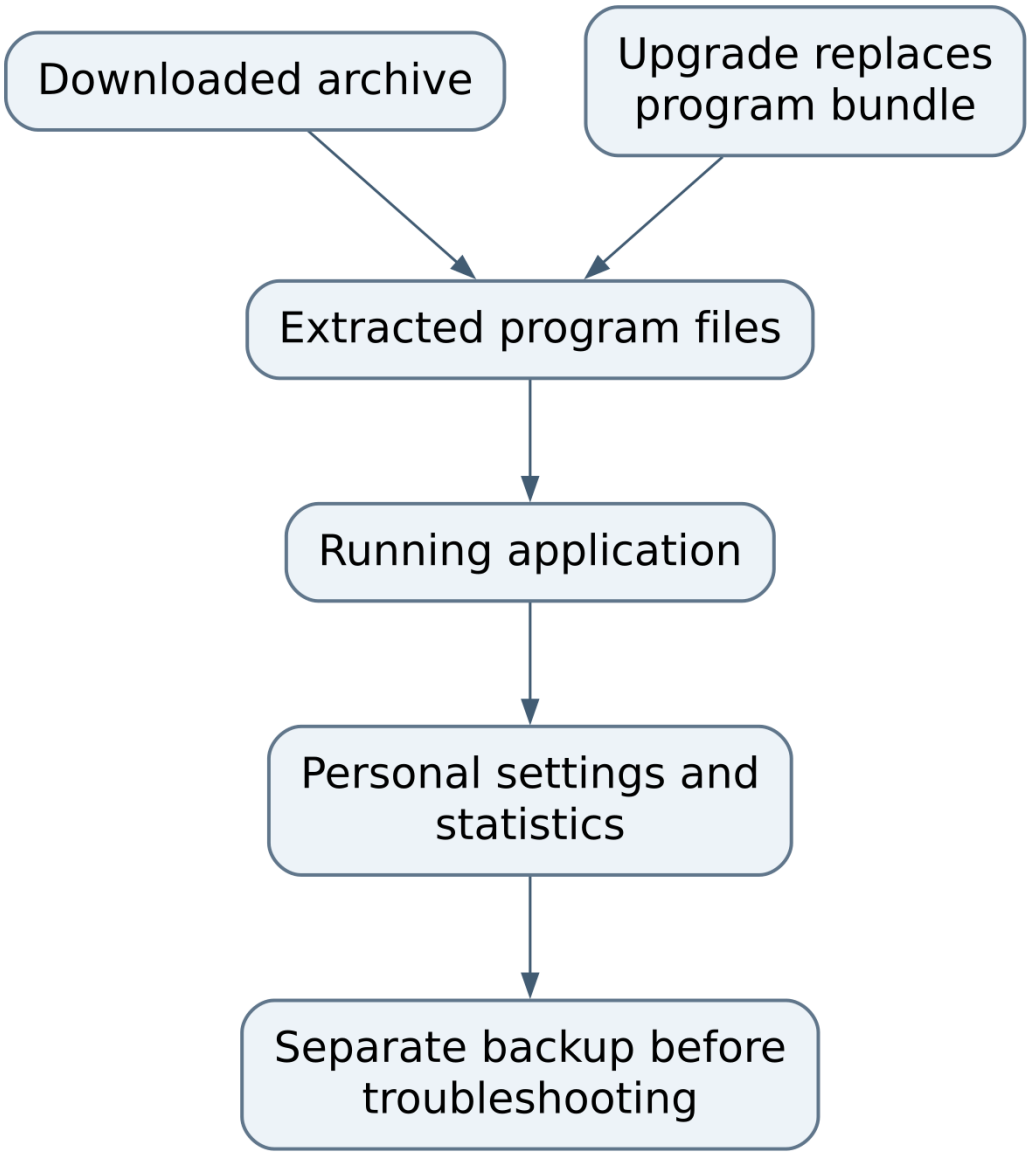


Figure 2 Installed files versus personal data

The application-to-data arrow describes documented macOS separation and a general preservation boundary; it does not invent unverified paths on other platforms. There is no database service, port or container defined here. A program closing is not a remote deployment shutdown.

What a support report should contain

Record OS and architecture, actual release tag, archive filename and digest, launch command, and the shortest reproducible symptom. Include readable terminal errors where available, removing account paths or personal content before publishing. Do not attach an entire personal-data directory merely because settings may be involved. Preserve originals privately for later investigation.

Local setup on NixOS and other platforms

Reviewing the repository

From the checkout, inspect `git status --short --branch`, `git ls-files`, and `README.md`. There is no `npm install`, Python environment or build command in this repository. Git and a text editor are enough to edit the public documentation. Keep release experiments and downloaded archives outside tracked source. A topic branch and a human-reviewed PR are the publication path for documentation changes.

Linux and NixOS installation

Download the Linux x86_64 archive and its matching `.sha256` file from the same actual release. In the download directory:

```
sha256sum -c ClassicArcade-linux-x86_64.tar.gz.sha256
tar -xzf ClassicArcade-linux-x86_64.tar.gz
steam-run ./ClassicArcade/ClassicArcade
```

The first command is a verification gate: if it fails, stop and check filenames, exact bytes and release identity. The last command is the README's NixOS launch path. `steam-run` supplies a compatibility environment for conventional Linux binaries; the repository does not provide a pinned Nix shell or installation instructions for that host tool. Its presence and the graphical environment must be established locally. Do not change the host configuration or install packages merely to complete a documentation review.

A non-NixOS Linux desktop uses `./ClassicArcade/ClassicArcade` according to the README and must meet its glibc requirement. A terminal reporting “file not found” for an existing launcher can reflect a missing interpreter or loader, not a missing game file. On NixOS, first confirm that the documented wrapper was used before deleting or redownloading unrelated files.

Windows and macOS

On Windows, extract the complete zip and launch `ClassicArcade.exe`. On macOS, extract the zip, move the app into Applications and open it normally. The documented checksum command on macOS is:

```
shasum -a 256 -c ClassicArcade-macos.zip.sha256
```

Do not recommend removing quarantine or bypassing operating-system security as routine installation instructions. If the app fails signature validation, collect the actual release and error and have the publisher investigate the package. This checkout contains no certificate, signing key or notarization log with which to repair it.

Evidence limits

These are the committed installation paths, not a new test result. No binary was launched during this handbook's preparation. Before publishing a release change, the source/release maintainer must establish real local platform evidence, including NixOS for the Linux package. A README edit alone cannot prove a new bundle works.

Player journeys and normal close

First launch

Choose the package for the computer's architecture and operating system, download it with any checksum companion, verify it, and extract it to a location where the whole application can remain together. On NixOS, open a terminal in the extraction parent and use the documented wrapper command. Keep the terminal available during the initial launch so an early failure remains visible.

After the application opens, use its own visible menus for game selection and settings. This repository does not document exact key bindings, game rules or an internal settings screen layout, so this handbook does not invent button names. A release should ideally ship accessible help for those tasks. If controls are unclear, preserve that as a concrete documentation gap with a screenshot and release version.

Upgrade without losing history

Close the running application before replacing program files. Retain the old archive and record its version until the new release has been checked. On macOS, back up the documented settings/statistics directory separately, then replace the app bundle. Avoid copying arbitrary old executable files into the new bundle: that creates a mixed version that cannot be diagnosed reliably.

For Windows or Linux, determine the real personal-data location from the source project's documentation or the running application's diagnostics before attempting a settings reset. The public README does not currently establish it. Preserve candidate data rather than delete it based on a guess. A new application directory can help distinguish a damaged installation from personal settings, but it does not necessarily isolate user data because the application may use a shared platform directory.

Normal close

Use the application's ordinary quit control or close its window. Wait for the process to exit before moving the bundle or restoring data. If launched in a terminal, confirm that the prompt returns. Do not assume an unresponsive window means it is safe to overwrite files while a process is still writing. This repository does not document the atomicity of settings/statistics writes, so conservative backup and complete process exit are appropriate.

Troubleshooting information to collect

Distinguish “archive will not extract,” “launcher cannot start,” “window opens then closes,” “sound or controls fail,” and “personal progress changed.” Those are different boundaries. Report the first reproducible failure and exact command instead of the broad statement “Linux is broken.” A reproducible support case should identify the latest working release too, when known.

Maintaining releases and documentation

Keeping the handbook useful

When code changes, update the chapter that explains its contract, the relevant diagram, the change recipe, and this pinned source register. Keep the Word and PDF editions synchronized with the Markdown chapters. Record what was actually tested, what remained unverified, and which saved data must survive an upgrade. A future maintainer should be able to trace an instruction to a file or a test without relying on a previous conversation.

Recipe publish clearer installation instructions

Read the current README and inspect an actual candidate release's asset names through the release owner's approved workflow. Make a table of documented package, checksum companion, extraction result and launcher. Correct only claims supported by that package. Keep `latest` links for convenience while retaining a named version and actual local verification in the PR evidence.

Test every relative handbook link locally and run `git diff --check`. A local source diff can verify spelling and destinations; it cannot verify a remote asset exists or a game launches. If remote links are being checked, actually inspect them rather than treating a Markdown parser's acceptance as network evidence. No remote release inspection was performed for this edition.

Recipe improve a NixOS failure report

Begin with an unchanged verified archive, record the wrapper command and collect the terminal error. Confirm that the executable remains in its complete extracted directory. Ask whether the failure occurs before a window appears, while opening graphics/audio, or during normal play. Reproduce under the source project's maintained environment before changing packaging. If a missing library is suspected, fix the release contract or documented environment deliberately; do not tell every player to copy random libraries beside the executable.

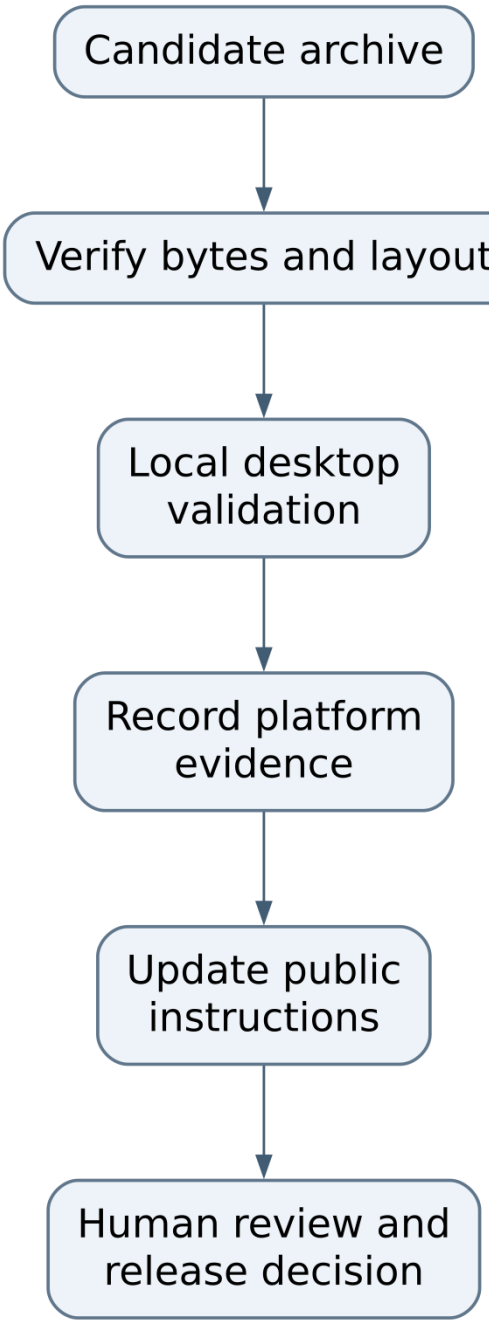


Figure 3 Recipe: improve a NixOS failure report

This is a maintenance lifecycle, not an existing automated workflow. This repository does not provide a release script. Routine hosted CI remains disabled under workspace policy; release/deployment actions require their own authorized human gate.

Recipe document personal data safely

Obtain the actual platform path, filenames, save trigger and schema/version behavior from the source implementation and local tests. Add a public guide that explains backup and restore using disposable test data. Prove that replacing the application preserves statistics, that a restored copy is read correctly, and that malformed data produces a useful error or safe fallback. Never publish a real player's complete files as a test fixture.

An executable bug fix belongs in the source repository. The public repo can track the symptom and corrected release version without disclosing private code. Keep owner-only implementation details out of public documentation; a public support procedure should describe observable behavior and supported remedies.

Known gaps and recovery

Checksum mismatch

Inspect first. Asset and checksum from the same release; exact filename
Safe next action. Redownload matching files; do not launch a mismatched archive

Missing files after extraction

Inspect first. Complete extracted directory and available disk space
Safe next action. Extract a verified archive into a fresh directory; preserve personal data

Existing Linux launcher reports missing file

Inspect first. Actual command and NixOS compatibility wrapper
Safe next action. Retry the documented `steam-run` path, then collect the error

Graphics window never opens

Inspect first. Desktop availability and terminal startup error
Safe next action. Diagnose graphics/runtime boundary in a local test; do not reset statistics

macOS security rejection

Inspect first. Exact release and operating-system message
Safe next action. Escalate package/signing investigation to publisher; retain verification evidence

Statistics appear reset

Inspect first. Actual platform data path and prior backup
Safe next action. Stop further writes and compare preserved copies; do not repeatedly reinstall

Latest link gives an unexpected build

Inspect first. Actual tag and downloaded digest
Safe next action. Pin the report to the named release rather than the moving link

Prioritized improvements

First, document actual Windows and Linux personal-data paths, save filenames and safe recovery behavior with source-backed verification. Acceptance is a disposable account test showing an upgrade and restore retain expected settings and statistics. Second, publish a platform validation record for each release, including NixOS launch, controls, sound, normal close and repeat launch. Acceptance is concrete recorded local results rather than a general Linux claim.

Third, make checksums and immutable version references consistent across all platform downloads. The root README supplies checksums for Linux and macOS but no Windows checksum link. Confirm the real release assets before adding one. Fourth, add a public controls/usage guide and an issue template focused on version, platform, archive integrity and reproducible behavior. These improve support without exposing private implementation.

Backup and rollback boundaries

Keep an untouched known-working archive and the associated digest. Back up personal data separately after the program has exited. On macOS, the path is explicitly documented; on other systems, establish it before a destructive cleanup. A rollback replaces complete program files with a known version and restores only data that version understands. Because schema compatibility is not documented here, do not promise that an older release can read data last written by a newer one.

There is no server or container to stop, no database volume in this repository, and no production configuration to restore from Git. The public documentation is recovered through ordinary Git history; binaries are recovered from verified release artifacts. If a release asset disappears, the public checkout alone cannot reconstruct it. Maintaining an authorized release archive belongs to the publisher's preservation process.

Source reference

This handbook describes committed source in OchsSoft/classic-arcade at `21cf8599dd68875ebd2042cf8e5e0f88c418cdac`. Paths below are relative to this repository. Source-defined behavior is distinguished from a fresh runtime verification; no application or game was launched to author these chapters. The existing quick operating guide remains available separately.

Reading the evidence

A source file explains what the program is designed to do. A test states an expectation and may exercise only one layer. A README describes an intended workflow and can lag implementation. When these disagree, inspect the exact revision and the executable code, reproduce the disagreement in a safe local test, and correct the relevant documentation with the change. Do not treat a passing unit test as proof of rendering, sound, external connectivity, deployment, or save recovery.

File and symbol index

File	Responsibility	Entry points and locations
README.md	Public product boundary, asset names, launch commands, platform and data assertions	L1 to L49